

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Event Management System

Field	Details
Project	Event Management System
Version	1.0
Authors	Jagriti Mohan (PES2UG24CS200); Karamsetty Aaryesh (PES2UG24CS215); Hitha Shree Suresh (PES2UG24CS196); L.A. Gokul (PES2UG24CS246)
Date	04-10-2026
Status	Final

Revision History

Version	Date	Author	Change Summary
1.0	04-10-2026	Team	Final Software Architecture and Design Specification for the Event Management System

Approvals

Role	Name	Signature / Date
Course Coordinator	Sheela Devi	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Event Management System (EMS), Version 1.0. It translates the requirements in the Event Management System SRS into an architectural structure, component responsibilities, security architecture, design interactions, API interfaces, error handling, monitoring considerations, and requirements traceability.

1.2 Scope

The architecture and design cover user registration and authentication, event creation and management, event discovery, event registration, payment handling, ticket generation, registration cancellation, participant management, notifications, administrative monitoring, performance, availability, scalability, audit logging, usability, and security.

1.3 Audience

Developers, QA engineers, system administrators, event organizers, users, and assessment evaluators.

1.4 Definitions

SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; STP – Software Test Plan; RTM – Requirements Traceability Matrix; FR – Functional Requirement; NFR – Non-functional Requirement; SR – Security Requirement; API – Application Programming Interface; RBAC – Role-Based Access Control; HTTPS – Hypertext Transfer Protocol Secure; TLS – Transport Layer Security; UI – User Interface; DB – Database.

2. Document Overview

2.1 How to Use This Document

The document describes the architecture first and then the detailed design. The architecture section defines goals, stakeholders, components, the chosen architecture pattern, technology/data considerations, risks, traceability, and security. The design section defines major interactions, APIs, error handling, monitoring, UX, and next steps.

2.2 Related Documents

- Event Management System Software Requirements Specification (SRS), Version 1.0, 06-09-2026.
- Event Management System Software Test Plan (STP), Version 1.0, 15-09-2026.
- Software Engineering Mini-Project Deliverables Part-1.

3. Architecture

3.1 Goals & Constraints

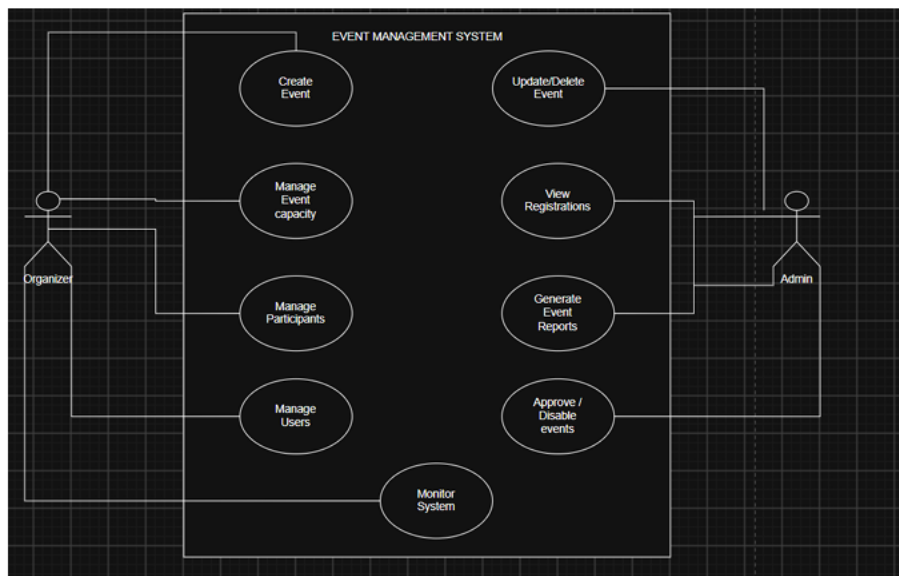
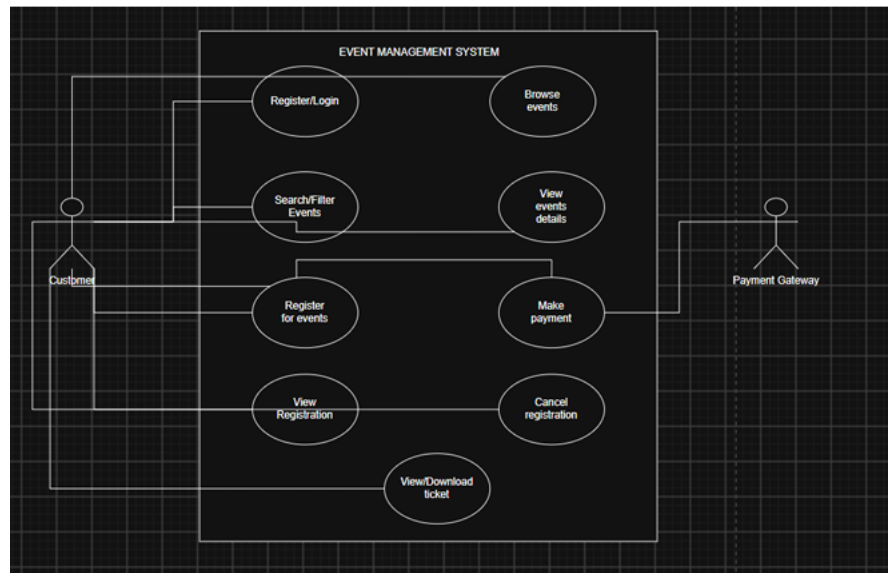
- Provide a modular web-based architecture for Users, Event Organizers, and Administrators.
- Separate presentation, application/business logic, and data responsibilities.
- Support event creation, discovery, registration, capacity control, payment, ticketing, participant management, notifications and administration.
- Protect user, event, registration and payment-related information.
- Support the SRS quality targets for response time, availability, scalability, auditability and usability.
- Online access and payment processing require network connectivity.
- Protected functions require authenticated users with appropriate roles.
- Payment processing depends on the configured payment gateway.

- User and registration data must be protected from unauthorized access.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Users	Security, event discovery, registration, payment, tickets and cancellation.
Event Organizers	Event creation/management, capacity, participants and reports.
Administrators	User/event management, event approval/disabling, monitoring and audit information.
Developers	Modularity, maintainability, clear interfaces and traceability.
QA Engineers	Requirement coverage, functional/non-functional testing and security validation.
System Administrators	Availability, monitoring, secure operation and auditability.
Assessment Evaluators	Compliance with SRS and SE Part-1 deliverables.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Responsibility
Web UI / Presentation	Responsive interface for login, event discovery, event details, registration, payment, tickets, organizer functions and administrator functions.
Authentication & RBAC	Account creation, login, authentication and role-based authorization for User, Event Organizer and Administrator.
Event Management	Create, update, delete, search, filter and display event information.
Registration	Create registrations, enforce event status/capacity and support eligible cancellation.
Payment	Process applicable registration payments through the configured payment gateway and handle payment outcomes.
Ticket	Generate registration confirmation and ticket details after successful registration.
Participant	Allow organizers to view and manage participant registrations for events they manage.
Notification	Generate configured registration confirmations and event notifications.
Admin	Support user management, event approval/disabling and review of system activity.
Database	Persist users, events, registrations, payments, tickets and audit information.
Audit Logging	Record important user, organizer, administrator, event, registration, payment and security-relevant actions with timestamps.

3.5 Chosen Architecture Pattern and Rationale

Layered Architecture is selected. The Presentation Layer handles user interaction; the Application/Business Layer implements authentication, event, registration, payment, ticket, participant, notification and administrative logic; and the Data Access Layer manages persistence. External payment and notification services are integrated through defined service interfaces.

This separation provides clear responsibilities, modularity, maintainability, easier testing and direct traceability from SRS requirements to components. It also matches the SRS product perspective, which identifies a web UI, application services, a database, and external payment/notification services.

3.6 Technology Stack & Data Stores

The SRS specifies a web application, server-side application environment, database server, payment gateway API, email/notification service, HTTPS and JSON communication. It does not finalize a specific framework/database product in the supplied SRS. Therefore, this SAD deliberately does not claim an unapproved technology version.

Area	Architecture Decision / Baseline
Frontend	Responsive web UI; the team's selected frontend framework shall implement the SRS UI requirements.
Backend	Server-side application environment exposing application services/APIs.
Database	Database server for users, events, registrations, payments, tickets and audit information.
API format	Structured JSON requests/responses where applicable.
Communication security	HTTPS with TLS 1.2 or higher.
Payment	Configured payment gateway API/sandbox.
Notifications	Configured email/notification service.
API testing	Postman, as specified in the STP.
Performance/load testing	JMeter or equivalent, as specified in the STP.

3.7 Risks & Mitigations

Risk	Mitigation
Payment gateway unavailable	Use sandbox/test interfaces where available and isolate external-service failures from application defects.
Notification service unavailable	Handle communication failure and show an appropriate status to the user.

Insufficient capacity/registration data	Prepare open, closed and full-capacity event scenarios.
Unauthorized access	Enforce authentication and role-based access control.
Security defects found late	Perform security validation alongside functional testing.
High concurrent load	Perform load/scalability testing and verify no transaction-data loss or critical failures.

3.8 Traceability to Requirements

SRS ID	Requirement	SAD Component	STP Test Case
EMS-F-001	Create account	Authentication & RBAC	TC-REG-01
EMS-F-002	Authenticate user	Authentication & RBAC	TC-AUTH-01
EMS-F-003	Update profile	Authentication & RBAC	TC-PROF-01
EMS-F-004	Create event	Event Management	TC-EVT-01
EMS-F-005	Update/delete event	Event Management	TC-EVT-02
EMS-F-006	Search/filter events	Event Management	TC-SRCH-01
EMS-F-007	View event details	Event Management	TC-EVT-03
EMS-F-008	Register for event	Registration	TC-REG-02
EMS-F-009	Enforce capacity/status	Registration	TC-REG-03
EMS-F-010	Process payment	Payment	TC-PAY-01
EMS-F-011	Generate ticket	Ticket	TC-TKT-01
EMS-F-012	View/cancel registration	Registration	TC-CAN-01
EMS-F-013	Manage participants	Participant	TC-PART-01
EMS-F-014	Send notifications	Notification	TC-NOTIF-01
EMS-F-015	Admin management	Admin	TC-ADMIN-01
EMS-NF-001	Response time	System	TC-PERF-01
EMS-NF-002	Availability	System/Operations	TC-OPS-01
EMS-NF-003	Scalability	System	TC-LOAD-01
EMS-NF-004	Audit logging	Audit Logging	TC-AUD-01
EMS-NF-005	Responsive UI	Web UI	TC-UX-01
EMS-SR-001	TLS communication	Security/Communication	TC-SEC-01
EMS-SR-002	Secure password storage	Authentication/Database	TC-SEC-02
EMS-SR-003	Role-based access control	Authentication & RBAC	TC-SEC-03
EMS-SR-004	Input validation	Application Layer	TC-SEC-04
EMS-SR-005	Security audit logging	Audit Logging	TC-SEC-05

3.9 Security Architecture

The security architecture implements the two security objectives in the SRS: protect user, event, registration and payment-related information from unauthorized access/disclosure; and ensure only authenticated and authorized users perform protected operations while security-sensitive actions remain traceable.

Security Requirement	Architectural Control
EMS-SR-001	All client-server communication uses HTTPS with TLS 1.2 or higher.
EMS-SR-002	Passwords are stored only as secure password hashes; plaintext passwords are not stored.
EMS-SR-003	RBAC restricts User, Event Organizer and Administrator functions.
EMS-SR-004	User-supplied input is validated and sanitized before processing/storage.
EMS-SR-005	Failed logins, privileged actions, event changes and registration/payment status changes are recorded in timestamped audit logs.

4. Design

4.1 Design Overview

The design separates user interaction from business services and persistence. Protected operations pass through authentication and authorization. Event registration checks event status and capacity before creating a registration. Applicable payments are handled through the configured payment gateway, followed by registration confirmation and ticket generation.

4.2 UML Sequence Diagrams

Sequence Diagram 1 — User registers for an event, including authentication, capacity/status validation, applicable payment, ticket generation and notification.

Figure 2. UML Sequence Diagram — User Event Registration

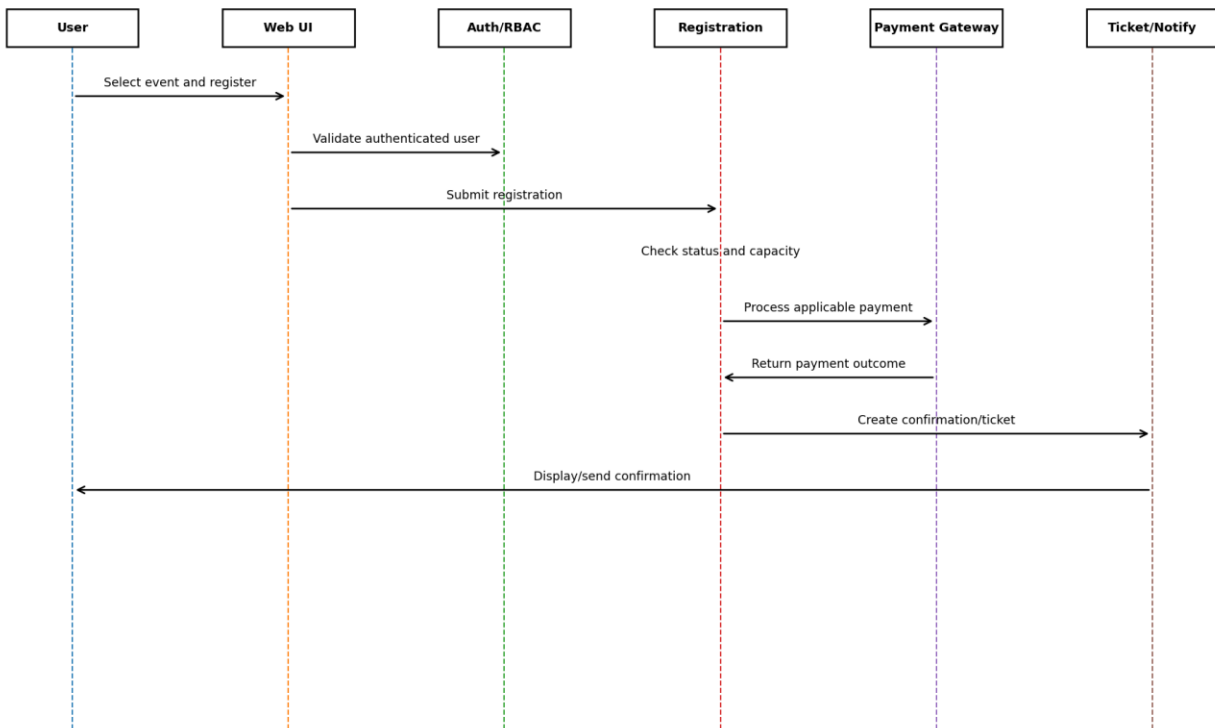
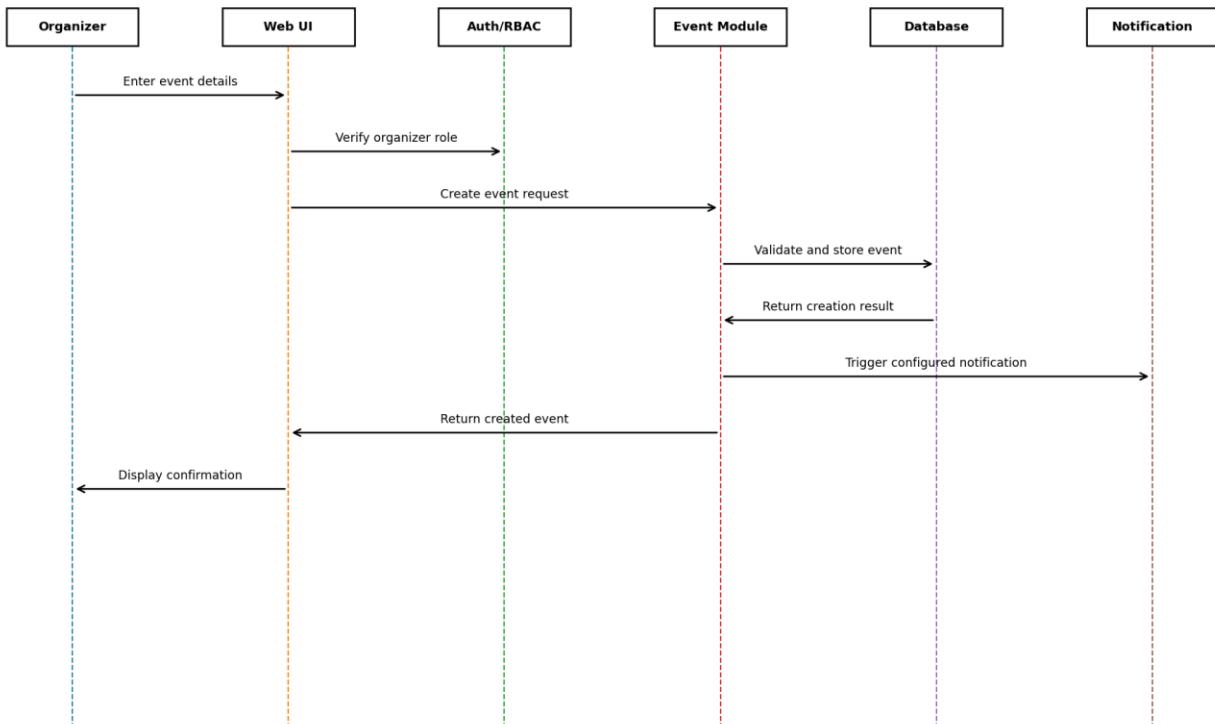


Figure 3. UML Sequence Diagram — Organizer Event Creation



4.3 API Design

These are design-level API interfaces derived from the SRS functions. Exact endpoint naming can be aligned with the implemented backend without changing the underlying requirements.

Component	Method	Endpoint	Purpose	Key Errors
Authentication	POST	/api/auth/register	Create user account	Invalid/missing registration data
Authentication	POST	/api/auth/login	Authenticate registered user	Invalid credentials
Events	POST	/api/events	Create event	Unauthorized/invalid data
Events	GET	/api/events	Search/filter events	Invalid filter
Events	GET	/api/events/{id}	View event details	Event not found
Registration	POST	/api/events/{id}/register	Register for event	Closed/full event
Registration	DELETE	/api/registrations/{id}	Cancel eligible registration	Invalid/ineligible cancellation
Payment	POST	/api/payments	Process applicable payment	Payment failure
Admin	PUT	/api/admin/events/{id}	Approve/disable event	Unauthorized admin action

4.4 Error Handling, Logging & Monitoring

Situation	Handling
Invalid credentials	Reject authentication and return a generic authentication error.
Unauthorized role	Reject the protected operation and record a security-relevant event where applicable.
Invalid/malicious input	Validate/sanitize and reject or safely handle input.
Event not found	Return an appropriate not-found status without sensitive information.
Event full/registration closed	Reject registration and display an appropriate message.
Payment failure	Record the payment outcome and return a clear failure status.
External service failure	Handle communication failure and display an appropriate status.
Unexpected application error	Return a standardized error response and log diagnostic information without sensitive data.

Monitoring covers response time, availability/uptime, load-test transaction success, audit-log retrieval and security-test status. The STP defines corresponding test cases for these quality attributes.

4.5 UX Design

- Responsive web interface on supported desktop and mobile screen sizes.
- Clear navigation for login, event discovery, event details, registration, payment, tickets, organizer functions and administrator functions.
- Clear validation and error messages on forms.
- Understandable status feedback for registration and payment.
- Consistent and accessible interaction patterns.

4.6 Open Issues & Next Steps

- Finalize the team's exact frontend/backend framework and database product if not already fixed outside the supplied SRS.
- Implement the components and interfaces described in this SAD.
- Configure the payment gateway sandbox and notification service.
- Implement and validate authentication, RBAC, input validation and audit logging.
- Execute the STP test cases and update the RTM with actual Pass/Fail/Blocked/Not Executed results.
- Perform performance, scalability, usability and security validation before final release.

5. Appendices

5.1 Glossary

Term	Meaning
API	Application Programming Interface
EMS	Event Management System

FR	Functional Requirement
NFR	Non-functional Requirement
RBAC	Role-Based Access Control
RTM	Requirements Traceability Matrix
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
STP	Software Test Plan
TLS	Transport Layer Security
UI	User Interface

5.2 References

- Event Management System SRS, Version 1.0, dated 06-09-2026.
- Event Management System Software Test Plan, Version 1.0, dated 15-09-2026.
- Software Engineering Mini-Project Deliverables Part-1.
- SAD template supplied for the Software Engineering mini-project.

5.3 Tools

- UML/component and sequence diagramming tool or draw.io equivalent.
- Postman for API testing.
- Browser developer tools/manual UI testing.
- JMeter or equivalent for performance/load testing.
- Document editor for SAD maintenance.